

M11 Teaching Book

Storage & File Formats Foundations - beginner-first, self-contained teaching

How to use this book

Watch the assigned visual first for mental imagery, then close it. The book must stand on its own. Work through explanation and demonstration before touching the independent task. A plausible-looking result is not evidence until you validate it against an independent control.

Boundary

M11 teaches physical storage and Delta concepts deeply enough to reason correctly. It deliberately does not turn into a Spark or production Delta implementation course. M12 introduces Spark/PySpark; M13 implements deeper Delta/lakehouse engineering.

Version baseline

PyArrow/Apache Arrow baseline: 25.0.x stable. Current Databricks documentation is used for Parquet/Delta semantics. Vendor-specific optimizations are clearly separated from open-format concepts.

DE11-01 - Row vs columnar storage; where Parquet fits

Prerequisite: M10 and G04 passed. You can work in WSL/Linux and Python. Spark is NOT required yet.

1. Outcome

Explain why analytical systems prefer columnar storage, identify where Parquet fits, and prove the difference between logical table grain and physical file layout.

2. Teacher explanation

A table is a logical collection of rows, but a storage format decides how those values are laid out on disk. Row-oriented layouts keep the fields for one record near each other, which is convenient when applications frequently read or update complete records. Columnar formats group values from the same column together. Analytical questions often need a few columns across many rows, so columnar layout can reduce unnecessary I/O and compress repeated values efficiently. Apache Parquet is a columnar file format, not a database and not a query engine. It stores typed columns plus metadata so engines can avoid reading data that cannot satisfy a query. The important beginner distinction is: grain tells you what one logical row means; file format tells you how the bytes are organized and accessed.

3. Working demonstration

Use orders.csv. State its grain: one row per order. Compare a question that needs every field for O1008 with a question that needs only region and amount for all orders. The first resembles row retrieval; the second is the kind of scan where columnar storage can help.

4. Why this matters

Data engineers are responsible not only for moving records but also for preserving correctness while storage layout, schemas and table versions change. This lesson connects the physical mechanism to an auditable business-data invariant.

Video first

[Databricks - The Parquet Format and Performance Optimization Opportunities](#)

Long-form visual: row vs columnar storage, Parquet physical layout, row groups/pages, compression, predicate pushdown, partitioning and small-file problems. Use only the lesson-relevant portion; no invented timestamp.

Preserved optional stage visual

[Databricks Academy - free training](#)

DE11-01 - Practice, Proof & Transfer

5. Guided reproduction

Run `parquet_probe.py` on the bundled Apache Parquet fixture. Confirm PAR1 at both ends. Then install PyArrow 25.0.* on your learner machine and rerun to expose schema/row-group metadata.

6. Independent task

Write a five-sentence explanation that separates logical row grain, columnar storage, Parquet, a query engine, and Delta Lake. Use one concrete analytics query from `orders.csv`.

7. Validate before trusting

Your explanation must not call Parquet a database. ``orders.csv`` has 12 rows, quantity 24, amount 2780.00, four regions and four dates.

8. Recover if weak

If row-vs-columnar still feels abstract, draw six columns and four rows on paper, then circle the values needed for ``SUM(amount) GROUP BY region``. Count how many cells are relevant.

9. Explain in your own words

Why can the same logical table be stored in CSV, Parquet, or a database table without changing its business grain?

10. Evidence to save

Save the command/code or decision artifact, observed output/control, one error or uncertainty, the repair you made, and a 60-90 second explain-back. Do not mark mastered from reading alone.

11. Protected review rule

Only after the independent attempt and evidence are saved may you open the matching M11 Review/Repair section. Close it before a changed retry.

Authoritative current references

[Databricks - Read and write Parquet files](#)

[Apache Arrow / PyArrow install - 25.0.x stable](#)

DE11-02 - Parquet schema, row groups and metadata

Prerequisite: DE11-01 mastered; PyArrow may now be installed for full local inspection.

1. Outcome

Read Parquet schema/metadata and explain row groups, column chunks, pages and the footer without confusing them with business partitions.

2. Teacher explanation

A Parquet file is not one undifferentiated block. Rows are divided into row groups. Within a row group, each column is stored as a column chunk; chunks contain encoded/compressed pages. The file footer stores metadata describing the schema and physical pieces. Engines use this metadata to plan reads. A row group is an internal horizontal slice inside a file. A directory partition such as `region=East/` is an external dataset-layout choice across files. They solve different problems. The footer is why the last bytes of a Parquet file matter: corrupting or truncating metadata can make an otherwise large file unreadable.

3. Working demonstration

The pure-Python probe reads the final four-byte PAR1 marker and the four-byte little-endian footer-length field immediately before it. It does not decode Thrift metadata; with PyArrow installed, ParquetFile exposes row-group count, total rows and schema safely.

4. Why this matters

Data engineers are responsible not only for moving records but also for preserving correctness while storage layout, schemas and table versions change. This lesson connects the physical mechanism to an auditable business-data invariant.

Video first

[Databricks - The Parquet Format and Performance Optimization Opportunities](#)

Long-form visual: row vs columnar storage, Parquet physical layout, row groups/pages, compression, predicate pushdown, partitioning and small-file problems. Use only the lesson-relevant portion; no invented timestamp.

Preserved optional stage visual

[Databricks Academy - free training](#)

DE11-02 - Practice, Proof & Transfer

5. Guided reproduction

Use the bundled Apache fixture. Record file size, footer metadata length, row-group count and schema when PyArrow is available. Point to the difference between physical row groups and your region directories from the layout lab.

6. Independent task

Create `orders_from_course.parquet` using `build_course_parquet.py`, which writes `row_group_size=4`. Predict the approximate row-group count before running, then verify it.

7. Validate before trusting

For 12 rows written with a row-group target of 4, expect 3 row groups in this small controlled file. Do not generalize this exact count to production writers.

8. Recover if weak

If PyArrow import fails, verify the active interpreter and run `python -m pip install "pyarrow==25.0.*"`; use the PAR1 probe meanwhile. Do not install Spark to solve a PyArrow problem.

9. Explain in your own words

What is the difference between a row group and a partition directory, and why does Parquet keep metadata in a footer?

10. Evidence to save

Save the command/code or decision artifact, observed output/control, one error or uncertainty, the repair you made, and a 60-90 second explain-back. Do not mark mastered from reading alone.

11. Protected review rule

Only after the independent attempt and evidence are saved may you open the matching M11 Review/Repair section. Close it before a changed retry.

Authoritative current references

[Databricks - Read and write Parquet files](#)

[Apache Parquet format specification](#)

DE11-03 - Projection, predicates and analytical reads

Prerequisite: DE11-02; you can inspect Parquet metadata.

1. Outcome

Use projection and predicates deliberately, and explain how metadata/statistics may reduce physical reading without promising impossible zero-I/O behavior.

2. Teacher explanation

Projection means requesting only the columns a query needs. Predicate filtering means expressing row conditions such as ``region = East`` or ``amount >= 250``. Columnar storage makes projection powerful because an engine can avoid decoding unrelated columns. Parquet statistics and indexes can sometimes let an engine skip row groups whose min/max or other metadata prove they cannot match a predicate. This is often called predicate pushdown or data skipping. It is an optimization, not a correctness rule: the query must still return the same logical result if no skipping is possible. Never validate a query only by saying it was faster.

3. Working demonstration

With PyArrow, read only ``region`` and ``amount`` from the course Parquet, then filter `amount >= 250`. Compare the logical result with the same condition evaluated from `orders.csv`.

4. Why this matters

Data engineers are responsible not only for moving records but also for preserving correctness while storage layout, schemas and table versions change. This lesson connects the physical mechanism to an auditable business-data invariant.

Video first

[Databricks - The Parquet Format and Performance Optimization Opportunities](#)

Long-form visual: row vs columnar storage, Parquet physical layout, row groups/pages, compression, predicate pushdown, partitioning and small-file problems. Use only the lesson-relevant portion; no invented timestamp.

Preserved optional stage visual

[Databricks Academy - free training](#)

DE11-03 - Practice, Proof & Transfer

5. Guided reproduction

Record selected columns, qualifying order IDs, qualifying row count and sum(amount). Keep a separate full-scan calculation as a correctness control.

6. Independent task

Choose another two-column projection and one predicate. Explain which physical work an engine might skip and which result control proves correctness regardless of optimization.

7. Validate before trusting

For amount ≥ 250 on the course data, qualifying orders are O1002, O1004, O1008, O1010, O1012; count 5 and amount 1610.00.

8. Recover if weak

If a filtered read returns unexpected rows, prove the predicate against CSV first. Separate a logic error from a storage/engine optimization issue.

9. Explain in your own words

Why is projection an I/O optimization while result reconciliation is a correctness check?

10. Evidence to save

Save the command/code or decision artifact, observed output/control, one error or uncertainty, the repair you made, and a 60-90 second explain-back. Do not mark mastered from reading alone.

11. Protected review rule

Only after the independent attempt and evidence are saved may you open the matching M11 Review/Repair section. Close it before a changed retry.

Authoritative current references

[Databricks - Read and write Parquet files](#)

[PyArrow Parquet API](#)

DE11-04 - Compression, small files and compaction

Prerequisite: DE11-03; you understand Parquet reads.

1. Outcome

Explain compression, diagnose the small-file problem, and design compaction without confusing fewer files with fewer logical rows.

2. Teacher explanation

Compression reduces bytes, while compaction reduces file count by combining many small files into fewer larger files. They are related but not identical. Parquet commonly compresses within column chunks; repeated values often compress well. A lake containing thousands of tiny files can perform badly even if each file is compressed because planners and object stores still pay listing/open/metadata overhead. Compaction should preserve the logical dataset while changing its physical layout. The validation therefore compares row counts, keys and control totals before and after compaction - not just file size.

3. Working demonstration

Run `layout_lab.py`. It deliberately creates 12 one-row CSV files and a single compacted.csv from the same 12 rows. This is a file-count demonstration only, not a claim that CSV compaction equals Delta OPTIMIZE.

4. Why this matters

Data engineers are responsible not only for moving records but also for preserving correctness while storage layout, schemas and table versions change. This lesson connects the physical mechanism to an auditable business-data invariant.

Video first

[Databricks - The Parquet Format and Performance Optimization Opportunities](#)

Long-form visual: row vs columnar storage, Parquet physical layout, row groups/pages, compression, predicate pushdown, partitioning and small-file problems. Use only the lesson-relevant portion; no invented timestamp.

Preserved optional stage visual

[Databricks Academy - free training](#)

DE11-04 - Practice, Proof & Transfer

5. Guided reproduction

Reconcile tiny-file rows to compacted rows and the original controls. Then inspect Parquet compression choices in current PyArrow/Databricks references.

6. Independent task

Write a production decision note: when would you compact, what would you measure, and what controls must remain invariant? Include one risk of compacting too aggressively.

7. Validate before trusting

The classroom layout has 12 tiny files and 1 compacted file while preserving the 12-row business dataset. Normal amount remains 2780.00.

8. Recover if weak

If you think "one huge file is always best," list concurrency, parallelism and rewrite-cost trade-offs. The goal is healthy file sizing, not minimum file count.

9. Explain in your own words

What is the difference between compression and compaction, and why can 10,000 tiny compressed files still be a problem?

10. Evidence to save

Save the command/code or decision artifact, observed output/control, one error or uncertainty, the repair you made, and a 60-90 second explain-back. Do not mark mastered from reading alone.

11. Protected review rule

Only after the independent attempt and evidence are saved may you open the matching M11 Review/Repair section. Close it before a changed retry.

Authoritative current references

[Databricks - Optimize data file layout](#)

[PyArrow Parquet compression](#)

DE11-05 - Partitioning strategy and over-partitioning

Prerequisite: DE11-04; physical layout concepts are clear.

1. Outcome

Choose a partitioning strategy from query patterns and cardinality, and identify over-partitioning before it creates tiny-file/metadata problems.

2. Teacher explanation

Dataset partitioning places records into directory/file groups based on a partition key, often a low-to-moderate-cardinality field used for pruning. Good partitioning reduces the candidate files for common queries. Bad partitioning can create excessive directories and tiny files. Partitioning by a high-cardinality identifier such as `order_id` is usually a poor choice for a small analytics table because almost every row creates its own partition. Date partitioning can work for time-bounded ingestion, but daily/hourly choices must match volume and retention. Modern lakehouse platforms may also offer clustering techniques that reduce the need for traditional partitions; treat partitioning as a design decision, not a ritual.

3. Working demonstration

The layout lab partitions 12 orders into four region directories. A hypothetical East-only query has one candidate region file instead of four region files. That does not prove Parquet row-group skipping; it demonstrates directory pruning.

4. Why this matters

Data engineers are responsible not only for moving records but also for preserving correctness while storage layout, schemas and table versions change. This lesson connects the physical mechanism to an auditable business-data invariant.

Video first

[Databricks - The Parquet Format and Performance Optimization Opportunities](#)

Long-form visual: row vs columnar storage, Parquet physical layout, row groups/pages, compression, predicate pushdown, partitioning and small-file problems. Use only the lesson-relevant portion; no invented timestamp.

Preserved optional stage visual

[Databricks Academy - free training](#)

DE11-05 - Practice, Proof & Transfer

5. Guided reproduction

Compare region, order_date and order_id as candidate partition keys. For each, calculate distinct values on the 12-row data and explain expected file-count pressure.

6. Independent task

Design partitioning for a 500M-row event table with frequent last-7-day queries and a very high-cardinality customer_id. Defend your first choice and name the metric that would make you revisit it.

7. Validate before trusting

Course data distinct counts: region=4, order_date=4, order_id=12. A partition-by-order_id plan creates the clearest over-partitioning warning here.

8. Recover if weak

If you cannot choose a partition key, start from the most common selective query boundaries and data volume. Never choose only because a column “looks categorical.”

9. Explain in your own words

Why can over-partitioning make a columnar lake slower even though partition pruning sounds beneficial?

10. Evidence to save

Save the command/code or decision artifact, observed output/control, one error or uncertainty, the repair you made, and a 60-90 second explain-back. Do not mark mastered from reading alone.

11. Protected review rule

Only after the independent attempt and evidence are saved may you open the matching M11 Review/Repair section. Close it before a changed retry.

Authoritative current references

[Databricks - When to partition tables](#)

DE11-06 - Lake vs warehouse vs lakehouse

Prerequisite: DE11-01..05; you understand file layout basics.

1. Outcome

Compare data lake, data warehouse and lakehouse by storage, governance, transaction and workload needs without treating one architecture as universally best.

2. Teacher explanation

A data warehouse traditionally emphasizes managed structured tables, SQL analytics, governance and optimized query serving. A data lake emphasizes low-cost scalable object storage for many data types, but a plain collection of files does not automatically provide table transactions or strong governance. A lakehouse combines open/scalable lake storage with table-management capabilities such as reliable transactions, schemas, metadata and SQL-friendly access. These are architecture patterns, not brands. A company can use more than one. The engineering question is which guarantees and workloads are required, who operates the system, and what cost/complexity is justified.

3. Working demonstration

Place the M11 retail data into three conceptual designs: raw object files, curated warehouse tables, and lakehouse tables over object storage. For each, state where transaction/table metadata lives and who enforces contracts.

4. Why this matters

Data engineers are responsible not only for moving records but also for preserving correctness while storage layout, schemas and table versions change. This lesson connects the physical mechanism to an auditable business-data invariant.

Video first

[Databricks - The Best Data Warehouse is a Lakehouse](#)

Visual comparison of warehouse/lake/lakehouse trade-offs and why the lakehouse pattern exists.

Preserved optional stage visual

[Databricks Academy - free training](#)

DE11-06 - Practice, Proof & Transfer

5. Guided reproduction

Complete an architecture matrix for storage cost, schema governance, BI serving, ML/file access, transaction support and operational complexity.

6. Independent task

Given a small company with 50 GB structured sales data and five BI users, argue both for and against introducing a lakehouse now. Make a recommendation based on need, not fashion.

7. Validate before trusting

Your answer must name trade-offs. "Lakehouse is newer so always better" is an automatic reasoning defect.

8. Recover if weak

If the terms blur together, separate physical storage (files/object store) from table-management/catalog/query layers.

9. Explain in your own words

What capability turns a collection of Parquet files into something closer to a managed table?

10. Evidence to save

Save the command/code or decision artifact, observed output/control, one error or uncertainty, the repair you made, and a 60-90 second explain-back. Do not mark mastered from reading alone.

11. Protected review rule

Only after the independent attempt and evidence are saved may you open the matching M11 Review/Repair section. Close it before a changed retry.

Authoritative current references

[Databricks - What is a data lakehouse?](#)

DE11-07 - Delta tables, transaction log and ACID concepts

Prerequisite: DE11-06; no Spark installation is required in M11.

1. Outcome

Explain Delta table fundamentals: Parquet data files plus a transaction log, atomic commits, consistent table versions, and why direct file mutation is unsafe.

2. Teacher explanation

Delta Lake adds a transaction-log layer around data files. Instead of deciding table state by listing whatever files happen to exist, readers reconstruct a committed table version from the log. A transaction commit records which files/actions become part of the next version. Atomicity means a transaction becomes visible as a complete committed state or not at all. Consistency and isolation depend on the transaction protocol, while durability means committed state survives failures according to the storage system guarantees. The log is therefore not a decorative audit file. Directly editing or deleting data/log files behind the table protocol can break the table's history and correctness.

3. Working demonstration

Run `delta_foundation_simulator.py`. It writes `version_0/1/2` snapshots plus numbered classroom log records. This is deliberately NOT a Delta engine; it exists to make versioned commit thinking visible before M13.

4. Why this matters

Data engineers are responsible not only for moving records but also for preserving correctness while storage layout, schemas and table versions change. This lesson connects the physical mechanism to an auditable business-data invariant.

Video first

[That Databricks Guy - Master Delta Lake Key Capabilities](#)

2026 visual demo covering Delta ACID, time travel, optimization, schema enforcement/evolution and related capabilities. Official docs below control exact current behavior.

Preserved optional stage visual

[Databricks Academy - free training](#)

DE11-07 - Practice, Proof & Transfer

5. Guided reproduction

Inspect log versions 0,1,2. Identify operation, row count and schema. Explain why a reader should move from one committed version to the next rather than combine half-written state.

6. Independent task

Draw a failure timeline where new data files exist but the transaction commit is not published. Explain what an atomic table protocol should expose to a reader.

7. Validate before trusting

Simulator versions are [0,1,2]. V0 has 4 rows / spend 820.0; V1 has 5 rows / spend 1050.0; V2 adds `segment` without changing row count.

8. Recover if weak

If you equate Delta with "Parquet plus a JSON folder," focus on the protocol: correctness comes from committed log semantics, not merely the file names.

9. Explain in your own words

Why is a transaction log more important than simply having Parquet files in a folder?

10. Evidence to save

Save the command/code or decision artifact, observed output/control, one error or uncertainty, the repair you made, and a 60-90 second explain-back. Do not mark mastered from reading alone.

11. Protected review rule

Only after the independent attempt and evidence are saved may you open the matching M11 Review/Repair section. Close it before a changed retry.

Authoritative current references

[Databricks - What is Delta Lake?](#)

DE11-08 - MERGE/upsert and current-state records

Prerequisite: DE11-07; versioned table state is clear.

1. Outcome

Reason about MERGE/upsert as matched-update plus unmatched-insert logic, prove current-state controls, and detect ambiguous source matches.

2. Teacher explanation

An upsert combines update and insert behavior. Delta `MERGE` expresses a match condition between source and target, then applies actions for matched and unmatched rows. The most important engineering work happens before the syntax: define the target grain and business key, then ensure the source does not contain multiple rows that ambiguously match the same target key unless your logic deliberately resolves them. Current Databricks guidance warns that multiple matching source rows can make a MERGE ambiguous. A successful command is not sufficient proof; reconcile expected updates, inserts, target row count and control totals.

3. Working demonstration

Simulator v0 has C1-C4. Updates change C2 from Gold/350 to Platinum/420 and insert C5/160. After upsert, target has C1-C5 and spend 1050.

4. Why this matters

Data engineers are responsible not only for moving records but also for preserving correctness while storage layout, schemas and table versions change. This lesson connects the physical mechanism to an auditable business-data invariant.

Video first

[That Databricks Guy - Master Delta Lake Key Capabilities](#)

2026 visual demo covering Delta ACID, time travel, optimization, schema enforcement/evolution and related capabilities. Official docs below control exact current behavior.

Preserved optional stage visual

[Databricks Academy - free training](#)

DE11-08 - Practice, Proof & Transfer

5. Guided reproduction

Write the four controls before running: matched keys, inserted keys, expected target row count, expected total spend. Then compare to simulator report.

6. Independent task

Create a paper source with two different updates for C2. Explain why the source is ambiguous and propose one deterministic pre-merge rule using event timestamp + tie-breaker.

7. Validate before trusting

Expected simulator MERGE: updated=[C2], inserted=[C5], rows=5, spend=1050.0. Multiple source records for one target key must be resolved before claiming safe current-state logic.

8. Recover if weak

If target rows unexpectedly multiply, stop and test key uniqueness on both target and intended source update grain before changing the MERGE clause.

9. Explain in your own words

Why can a syntactically valid MERGE still be logically unsafe?

10. Evidence to save

Save the command/code or decision artifact, observed output/control, one error or uncertainty, the repair you made, and a 60-90 second explain-back. Do not mark mastered from reading alone.

11. Protected review rule

Only after the independent attempt and evidence are saved may you open the matching M11 Review/Repair section. Close it before a changed retry.

Authoritative current references

[Databricks - MERGE into Delta](#)

DE11-09 - Schema enforcement, controlled evolution and compatibility

Prerequisite: DE11-08; target grain and upsert rules are explicit.

1. Outcome

Distinguish schema enforcement from schema evolution and classify additive, rename/drop and type changes by compatibility risk.

2. Teacher explanation

Schema enforcement rejects or blocks writes that violate the table's expected structure or types. Schema evolution is a controlled process for accepting intended structural change. The safest beginner pattern is “fail closed, inspect, then evolve deliberately,” not “accept any new schema automatically.” Additive nullable columns are often easier to accommodate than incompatible type changes. Renames and drops can break downstream consumers even when the storage engine technically supports them. Current Databricks documentation describes separate mechanisms for additions, column mapping, type widening and other changes; supported does not mean risk-free. A data contract should state owner, expected schema, compatibility rule, rollout plan and validation.

3. Working demonstration

Simulator v2 adds `segment` to the v1 schema. It is a controlled additive change. Existing five rows remain five rows; each gets a deterministic derived segment.

4. Why this matters

Data engineers are responsible not only for moving records but also for preserving correctness while storage layout, schemas and table versions change. This lesson connects the physical mechanism to an auditable business-data invariant.

Video first

[That Databricks Guy - Master Delta Lake Key Capabilities](#)

2026 visual demo covering Delta ACID, time travel, optimization, schema enforcement/evolution and related capabilities. Official docs below control exact current behavior.

Preserved optional stage visual

[Databricks Academy - free training](#)

DE11-09 - Practice, Proof & Transfer

5. Guided reproduction

Classify four hypothetical changes: add nullable promo_code, rename amount to net_amount, widen INT to DOUBLE, change date to free-text STRING. Write consumer impact and migration evidence for each.

6. Independent task

Design a schema-change approval checklist with at least one producer check, one consumer check, rollback/replay plan and ownership field.

7. Validate before trusting

V2 schema is [id,name,tier,spend,segment] and v2_rows remains 5. Row-count stability alone is not enough; verify schema and key/control totals too.

8. Recover if weak

If “schema evolution” sounds like permission to ignore drift, rewrite the rule as: detect -> classify -> approve -> migrate -> validate -> monitor.

9. Explain in your own words

What is the difference between enforcement and evolution, and why can an additive column still break a consumer?

10. Evidence to save

Save the command/code or decision artifact, observed output/control, one error or uncertainty, the repair you made, and a 60-90 second explain-back. Do not mark mastered from reading alone.

11. Protected review rule

Only after the independent attempt and evidence are saved may you open the matching M11 Review/Repair section. Close it before a changed retry.

Authoritative current references

[Databricks - Schema evolution](#)

DE11-10 - Time travel, retention, VACUUM safety and medallion checkpoint

Prerequisite: DE11-01..09 mastered.

1. Outcome

Use version history/time travel conceptually, explain retention/VACUUM risk, and defend a bronze-silver-gold medallion checkpoint without deleting evidence prematurely.

2. Teacher explanation

Time travel means querying a prior committed table version or timestamp while its required files/log history are still retained. It is useful for debugging, audit comparison and reproducibility. Retention policies decide how long historical data files remain available. VACUUM removes files no longer referenced by recent table versions; after old files are deleted, sufficiently old time travel can stop working. Current Databricks guidance keeps a safety check around short retention and recommends at least seven days in normal circumstances. Do not disable safety checks as a beginner shortcut. Medallion architecture is a quality layering pattern: bronze preserves landed/raw evidence, silver applies validated/conformed transformations, and gold serves business-ready outputs. Promotion between layers should be proven by contracts and reconciliations, not folder names.

3. Working demonstration

Simulator proves time-travel difference: C2 is Gold in v0 and Platinum in current v2. Its VACUUM action is DRY_RUN_ONLY on purpose. No historical file is deleted in this module.

4. Why this matters

Data engineers are responsible not only for moving records but also for preserving correctness while storage layout, schemas and table versions change. This lesson connects the physical mechanism to an auditable business-data invariant.

Video first

[That Databricks Guy - Master Delta Lake Key Capabilities](#)

2026 visual demo covering Delta ACID, time travel, optimization, schema enforcement/evolution and related capabilities. Official docs below control exact current behavior.

Preserved optional stage visual

[Databricks Academy - free training](#)

DE11-10 - Practice, Proof & Transfer

5. Guided reproduction

Write a retention decision for a daily table with 10-day incident investigations and 7-day late reprocessing. Then map the retail source through bronze/silver/gold with one validation at each boundary.

6. Independent task

Explain why running VACUUM aggressively immediately after every job could damage debugging/recovery. Then design a medallion promotion checklist for an unseen dataset.

7. Validate before trusting

Time-travel control: v0 C2 tier=Gold; current v2 C2 tier=Platinum. VACUUM remains dry-run only. Your medallion checklist must preserve raw evidence and include row/key/control reconciliation.

8. Recover if weak

If time travel and backups seem identical, separate table-version history from an independent backup/disaster-recovery copy. They solve overlapping but different failure cases.

9. Explain in your own words

How can retention policy change the usefulness of time travel, and what must be true before bronze data is promoted to silver?

10. Evidence to save

Save the command/code or decision artifact, observed output/control, one error or uncertainty, the repair you made, and a 60-90 second explain-back. Do not mark mastered from reading alone.

11. Protected review rule

Only after the independent attempt and evidence are saved may you open the matching M11 Review/Repair section. Close it before a changed retry.

Authoritative current references

[Databricks - Delta tutorial/history](#)

[Databricks - VACUUM](#)